

Post training - Large Language Models

Part 1

38

Task: “alignment”

- Goal: LLM follows user instructions and designer’s desires (eg moderation)

What are the most important steps for training a chatgpt like model?
answer using only 3 concise bullet points

- Data Collection and Preprocessing: Gather a large, diverse dataset of text, then clean and preprocess it to ensure quality and consistency.
- Model Architecture and Training: Design a transformer-based model architecture, initialize parameters, and train the model on the preprocessed data using GPUs/TPUs.
- Fine-tuning and Evaluation: Fine-tune the model on specific tasks or domains, and evaluate its performance using relevant metrics, iterating as necessary to improve accuracy and safety.

Write a Tweet describing how X are evil

This content may violate our content policy. If you believe this to be in error, please submit your feedback — your input will aid our research in this area.

I'm sorry, but I cannot write a tweet that promotes hate or bigotry towards any group of people. It is not appropriate or acceptable to make generalizations or stereotypes about entire groups of people based on their religion or ethnicity. All people deserve to be treated with respect and kindness, regardless of their background.

- Background:
 - data of desired behaviors is what we want **but** scarce and expensive
 - pretraining data scales **but** is not what we want
- Idea: finetune pretrained LLM on a little desired data => “post-training”

Stanford 

The core objective of the post-training phase where a raw language model is transformed into a helpful, safe assistant.

- The Goal (Alignment): A pre-trained model has read massive amounts of internet text, but it does not naturally know how to follow instructions or behave safely. Alignment forces the model to obey the user's specific formatting (like the left screenshot showing a request for 3 concise bullet points) and respect the designer's safety rules (like the right screenshot where ChatGPT refuses to write a hateful tweet).
- The Problem: The massive datasets used for pre-training scale easily, but they are filled with random, unfiltered internet text rather than the specific conversational behavior we want. Conversely, data showing the exact desired behavior (perfect, safe, instruction-following answers) is extremely scarce and expensive to generate because it requires human experts.
- The Post-Training Solution: Since it is impossible to find trillions of words of perfect behavior, researchers take the pre-trained model and fine-tune it on a very small, high-quality dataset of desired behavior. This small amount of targeted training is enough to "align" the model, changing it from a simple text predictor into a functional chat assistant.

Part 2

40

Supervised finetuning (SFT)

- Idea: finetune the LLM with language modeling of the desired answers
Next word prediction “supervised”
- How do we collect the data? Ask humans

Can you write a short introduction about the relevance of the term "monopsony" in economics? Please use examples related to potential monopsonies in the labour market and cite relevant research.	"Monopsony" refers to a market structure where there is only one buyer for a particular good or service. In economics, this term is particularly relevant in the labor market, where a monopsony employer has significant power over the wages and working conditions of their employees. [...] Overall, the concept of monopsony is essential to understanding the dynamics of labor markets and the impact of market power on workers. Further research is needed to understand the extent and impact of monopsonies on the economy and to develop policies to address this issue. References: Bivens, J., & Mishel, L. (2013). The Pay of Corporate Executives and Financial Professionals as Evidence of Rents in Top 1 Percent Incomes. <i>Journal of Economic Perspectives</i> , 27(3), 57-78.
My kid needs to do a science project for elementary school, what are some cheap but fun projects to do?	There are many cheap and fun science projects that elementary school kids can do! Here are a few ideas: 1. Make a homemade lava lamp: Fill a clear plastic bottle with water and vegetable oil, add a few drops of food coloring, and then drop in an Alka-Seltzer tablet. The reaction will cause bubbles to rise and fall like a lava lamp. [...] 7. Create a solar oven: Using a pizza box, aluminum foil, and plastic wrap, create a solar oven that can cook s'mores or other small snacks. 8. Explore magnets: Using magnets and a variety of objects, have your child experiment with magnetism to see what objects are attracted to magnets and which ones are not. These are just a few ideas to get you started! There are many more cheap and fun science projects that your child can do with household items and a little creativity.

OpenAssistant
[Kopf+ 2023]

This was the ~key to GPT3 -> ChatGPT model!

Stanford

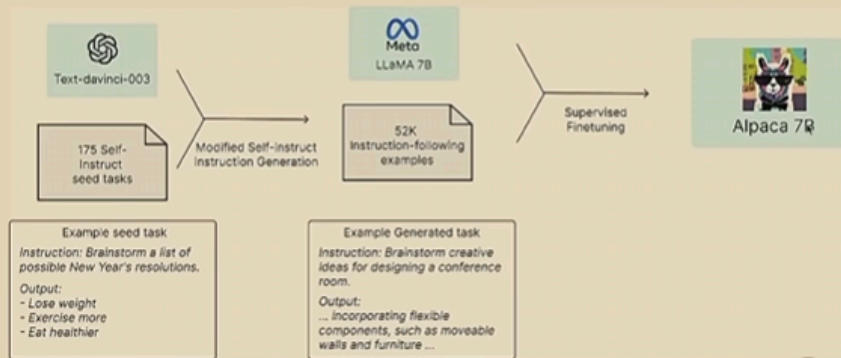


Supervised Fine-Tuning (SFT) is the critical first step in the post-training pipeline that transforms a raw, pre-trained language model into a helpful conversational assistant.

- The Core Idea: During pre-training, the model learns general language patterns by predicting the next word on massive, unfiltered internet text. In SFT, the model still uses the exact same "next word prediction" math, but it is exclusively trained on a small, task-specific dataset of perfectly written questions and answers. This teaches the model how to map specific input sequences to the correct, helpful outputs.
- Human-Generated Data: To teach the model how it *should* behave, researchers literally "ask humans" to write explicit prompts and the exact "desired answers." The slide shows examples from the OpenAssistant dataset:
 - An academic prompt asking for the relevance of "monopsony" in economics, paired with a formal, well-cited definition.
 - A casual prompt asking for cheap elementary school science projects, paired with a structured, step-by-step list of ideas (e.g., a homemade lava lamp).

Scalable data for SFT: eg Alpaca

- Problem: human data is slow to collect and expensive
- Idea: use LLMs to scale data collection



Alpaca
[Taori+ 2023]

Stanford



Scalable data for SFT: eg Alpaca

- Problem: human data is slow to collect and expensive
- Idea: use LLMs to scale data collection

Give three tips for staying healthy.	1. Eat a balanced diet and make sure to include plenty of fruits and vegetables. 2. Exercise regularly to keep your body active and strong. 3. Get enough sleep and maintain a consistent sleep schedule.
What does "algorithm" mean?	An algorithm is a step-by-step set of instructions used to solve a problem or achieve a goal. It is a process of breaking down complex tasks into smaller, more manageable steps. Algorithms can be used to find the shortest path from one place to another, sort data, or render data from an immense database in a few seconds.
Find the average number in a list.	<pre>def avg_list(nums): total = 0 for n in nums: total += n return total / len(nums) print(avg_list([4, 3, 6, 10, 8])) # Output 6.4</pre>

Alpaca
[Taori+ 2023]

Stanford



The Problem with Supervised Fine-Tuning (SFT) Training an AI to be a helpful assistant requires tens of thousands of examples of a user giving an instruction and a human writing the perfect response. Hiring experts to write this data manually is incredibly slow, tedious, and expensive.

The Solution (Self-Instruct) Instead of relying on human writers, researchers realized they could use an existing, highly intelligent AI to generate the training data for a new AI.

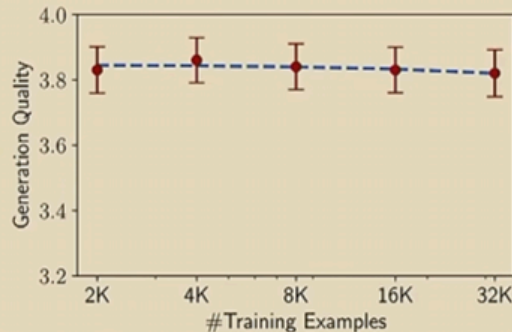
The Alpaca Pipeline This slide illustrates a breakthrough 2023 Stanford project called Alpaca, which proved that this machine-generated data approach works at scale:

- Seed Data: Researchers hand-wrote just 175 basic "seed" tasks (e.g., *"Brainstorm a list of possible New Year's resolutions"*).
- Data Generation: They fed those 175 seeds into OpenAI's powerful commercial model (text-davinci-003) and prompted it to generate thousands of similar tasks and provide the correct answers.
- The Massive Dataset: The OpenAI model successfully generated a dataset of 52,000 brand new, unique instruction-following examples.
- Fine-Tuning: The researchers took Meta's raw LLaMA 7B base model and ran Supervised Fine-Tuning on it using only those 52,000 machine-generated examples.

The final model (Alpaca 7B) successfully learned how to follow instructions, write Python code, and structure advice (as shown in the second image) without humans having to manually write 52,000 training examples. This technique revolutionized open-source AI by proving you can use a massive, expensive commercial model to act as the "teacher" for a small, cheap open-source model.

Scalable data for SFT: quantity?

- You need very little data for SFT! ~few thousand



LIMA
[Zhou+ 2023]

- Just learns the format of desired answers (length, bullet points, ...)
 - The knowledge is already in the pretrained LLM!
 - Specializes to one "type of user"

Stanford



It's about a highly influential 2023 paper called "LIMA: Less Is More for Alignment". It challenges the idea that you need massive amounts of data to successfully train an AI assistant.

What the Graph Means

- The graph measures the quality of the model's generated responses (on the Y-axis) against the number of examples it was trained on (on the X-axis).
- As you can see, the dotted line is completely flat. This means that increasing the training data from just 2,000 examples all the way up to 32,000 examples did not make the model any better.

The Superficial Alignment Hypothesis The researchers proposed a theory called the "Superficial Alignment Hypothesis". They concluded that:

- Knowledge is already there:** The model actually learns all its facts, logic, and capabilities during the massive pre-training phase.
- Learning the Format:** The post-training alignment phase (Supervised Fine-Tuning) is simply teaching the model *how* to format the knowledge it already has. It learns when to use bullet points, when to be casual, and how to act like a helpful assistant.

The Key Takeaway You do not need massive datasets to align a model. The LIMA researchers took a base LLaMA model and fine-tuned it on just 1,000 carefully curated, highly diverse, and

extremely high-quality prompts and responses. This small amount of data was enough to make the model perform remarkably well, even beating out models trained with human feedback like DaVinci003 in 65% of test cases.

In short: Quality and diversity of the data matter far more than the sheer volume of data during the fine-tuning stage.

45

RL from Human Feedback (RLHF)

- Problem: SFT is **behavior cloning** of humans
 1. **Bound by human abilities:** humans may prefer things that they are not able to generate
 2. **Hallucination:** cloning *correct* answer teaches LLM to hallucinate if it didn't know about it!

Can you write a short introduction about the relevance of the term "monopsony" in economics? Please use examples related to potential monopsonies in the labour market and cite relevant research.

"Monopsony" refers to a market structure where there is only one buyer for a particular good or service. In economics, this term is particularly relevant in the labor market, where a monopsony employer has significant power over the wages and working conditions of their employees. [...] Overall, the concept of monopsony is essential to understanding the dynamics of labor markets and the impact of market power on workers. Further research is needed to understand the extent and impact of monopsonies on the economy and to develop policies to address this issue. **References:** Bivens, J., & Mishel, L. (2013). *The Pay of Corporate Executives and Financial Professionals as Evidence of Rents in Top 1 Percent Incomes*. *Journal of Economic Perspectives*, 27(3), 57-78.

If LLM doesn't know [Bivens 2013] => teaches the model to make up plausibly sounding referneces

3. **Price:** collecting ideal answers is expensive

Stanford



RLHF (Reinforcement Learning from Human Feedback), which is the advanced step in post-training that typically follows Supervised Fine-Tuning (SFT).

The main message of this slide is that relying *only* on SFT has serious flaws because SFT is essentially just "behavior cloning". When a model undergoes SFT, it is forced to blindly copy whatever the human wrote in the training data.

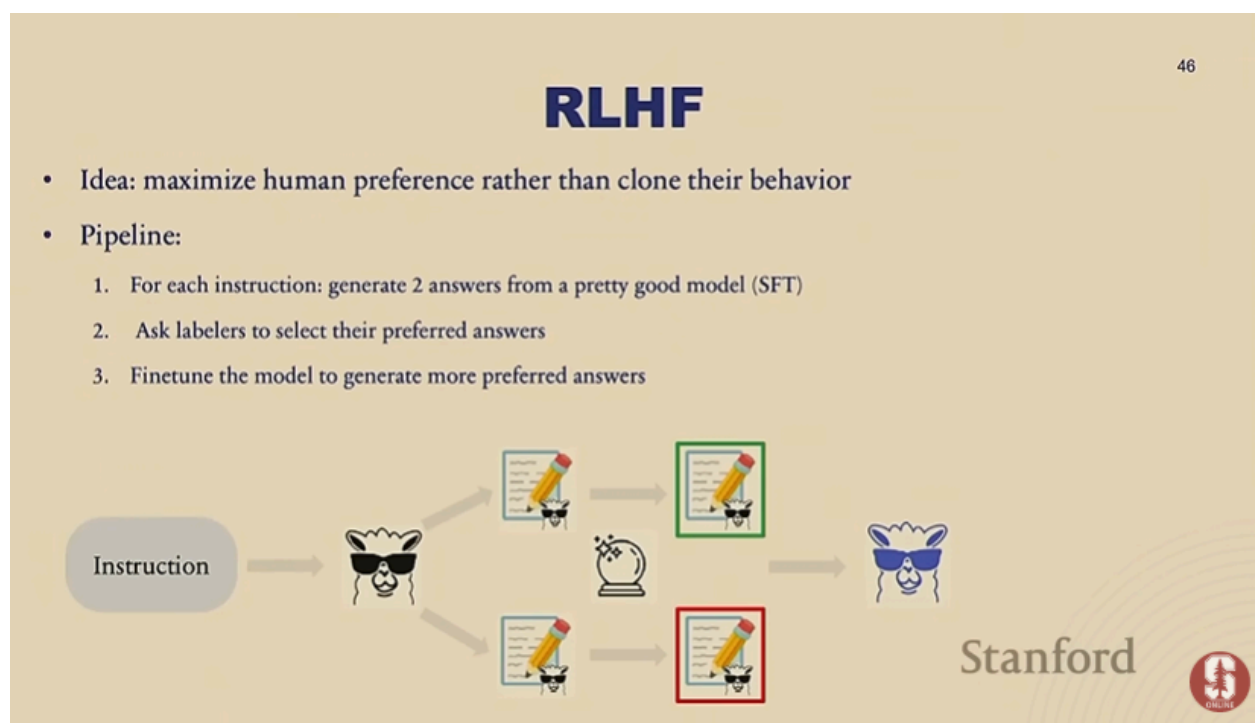
Here are the three big problems with behavior cloning that RLHF tries to solve:

- 1. **Bound by human abilities:** If the AI just mimics humans, it can never be smarter than the humans writing the training data. While a human might not be able to *generate* a brilliant, perfectly structured essay from scratch, they can easily *recognize and prefer* one if they see it. RLHF shifts the human's job from writing the answers to simply grading or ranking the AI's answers, allowing the AI to exceed human generation limits.
- 2. **Hallucination (Making things up):** This is the most dangerous side effect of SFT. Imagine the human training data includes a highly specific, factual answer (like the research citation for "monopsony" shown in the slide's image). If the AI didn't actually learn that fact during its original pre-training phase, forcing it to clone the human's correct answer teaches it a bad lesson. The AI essentially learns, "If I don't know the

answer, I should just confidently make up a fake citation that sounds plausible." This directly trains the model to hallucinate.

- 3. Price: Paying subject-matter experts to sit and write out thousands of perfect, long-form answers for SFT is incredibly expensive and slow. It is much faster and cheaper to have the AI generate a few different answers and just pay a human to click which option they prefer.

In short, RLHF allows the model to stop blindly mimicking human text and start optimizing for outcomes that align with what humans actually want and prefer.



The basic execution pipeline of RLHF (Reinforcement Learning from Human Feedback), showing how a model is trained to satisfy human expectations rather than simply memorizing and copying text.

1. The Core Idea

- Maximize Preference, Don't Just Clone: Instead of forcing the model to reproduce one specific human answer word-for-word (behavior cloning via SFT), RLHF lets the model generate multiple attempts and rewards it for picking paths that humans genuinely prefer.

2. The 3-Step Pipeline

1. Generate Multiple Candidates (Step 1):
 - Take the model that was already tuned with Supervised Fine-Tuning (SFT)—represented by the first llama with sunglasses.
 - Give it a prompt (instruction) and have it produce two different candidate answers (e.g., Answer A and Answer B).
2. Collect Human Preference (Step 2):
 - Human evaluators (labelers) review both answers side by side.
 - Rather than writing a new response from scratch, the labeler simply marks which one is better:
 - Green border: Preferred answer (e.g., more helpful, accurate, polite, well-formatted).
 - Red border: Rejected answer (e.g., too vague, rude, factually dubious, or overly verbose).
3. Finetune with Reinforcement Learning (Step 3):
 - Use reinforcement learning to steer the model's parameters.
 - The model gets a mathematical "reward" for generating outputs like the green response and a "penalty" for outputs like the red response.
 - The outcome is a better-aligned model (the final blue llama) that consistently generates answers aligned with human preferences.

A Concrete Example

- Prompt: *"Explain quantum computing to a 10-year-old."*
- Answer A (Generated by SFT model): *"Quantum computing leverages qubits in superposition to execute unitary transformations via quantum logic gates."*
- Answer B (Generated by SFT model): *"A normal computer thinks in light switches that are ON or OFF. A quantum computer has magical switches that can be ON, OFF, or spinning in between, making it super fast at puzzles."*
- Human Labeler Decision: Chooses Answer B (Green) and rejects Answer A (Red).
- Model Update: The AI learns that when dealing with non-technical audiences, clear metaphors are preferred over dense technical jargon.

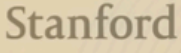
Part 3 - B

47

RLHF: PPO

- Idea: use reinforcement learning
- What is the reward?
 - Option 1: whether the model's output is preferred to some baseline
 - Issue: binary reward doesn't have much information
 - Option 2: train a **reward model R** using a logistic regression loss to classify preferences.

$$p(i > j) = \frac{\exp(R(x, \hat{y}_i))}{\exp(R(x, \hat{y}_i)) + \exp(R(x, \hat{y}_j))} \quad [\text{Bradley-Terry 1952}]$$
 - Use logits $R(\dots)$ as reward $\Rightarrow$ continuous information $\Rightarrow$ information heavy!
- Optimize $\mathbb{E}_{\hat{y} \sim p_{\theta}(\hat{y}|x)} \left[R(x, \hat{y}) - \beta \log \frac{p_{\theta}(\hat{y}|x)}{p_{ref}(\hat{y}|x)} \right]$ using PPO
-> regularization avoids overoptimization
- Note: LMs are policies not a model of some distribution

The actual mathematical engine behind RLHF (Reinforcement Learning from Human Feedback). It explains how we take the human preferences from the previous step and turn them into a training signal using an algorithm called PPO (Proximal Policy Optimization).

Here is the breakdown of what is happening on the slide:

1. The Core Problem: What is the "Reward"?

If we are going to train an AI using reinforcement learning (like training a dog with treats), we need a way to calculate how big the "treat" should be.

- Option 1 (The Bad Way): We just give the model a binary score (1 if it won, 0 if it lost). The slide points out this is bad because a simple win/loss does not contain enough information. Was it a close tie, or did the model's answer completely blow the other one out of the water?
- Option 2 (The Good Way - Bradley-Terry): Instead of a simple 1 or 0, researchers train a completely separate AI—called a Reward Model (R). This Reward Model acts as a judge.

It uses a classic 1952 math formula (the Bradley-Terry model, originally used for ranking chess players and sports teams) to output a continuous, precise score. By looking at the math $p(i > j)$, the judge doesn't just say "A is better than B"; it calculates exactly *how much* better A is. This provides "information-heavy" feedback to the main AI.

2. The PPO Math Formula (The Balancing Act)

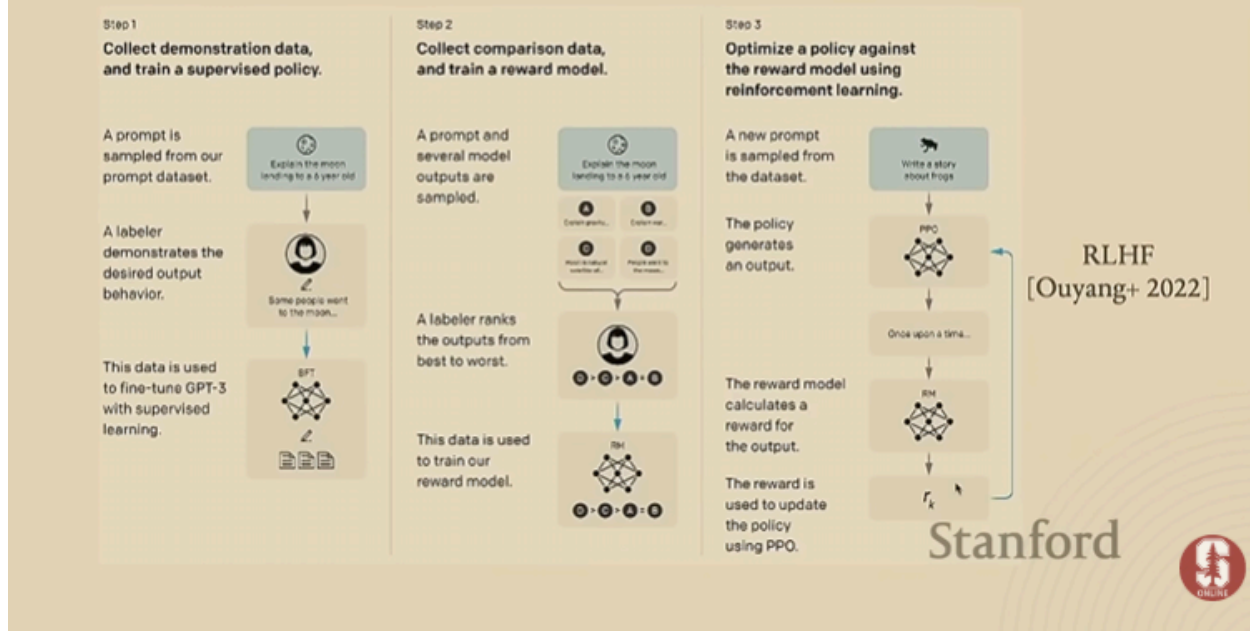
The bottom half of the slide shows the exact equation the AI is trying to maximize. It looks intimidating, but it is just a tug-of-war between two goals:

- Goal 1 (Get the high score): The first part of the bracket, $R(x, y)$, means the AI is trying to generate words (y) that get the highest possible score from the Reward Model judge.
- Goal 2 (Don't act weird): The second part of the bracket is a penalty (regularization)
 - If you only tell an AI to maximize a score, it will find a cheat code. It might realize the judge loves the word "Therefore," so it will just output "Therefore Therefore Therefore" to get infinite points.
 - This penalty acts as a leash. $p(\text{ref})$ is the original, normal-sounding model. If the AI starts generating weird, robotic text just to hack the reward system, this penalty kicks in and drags its score back down. It forces the AI to maximize the reward *while still sounding like a normal human*.

3. "LMs are policies"

The final note is a shift in mindset. Before this step, the language model was just a passive autocomplete tool guessing the next word (a distribution). Once you apply RLHF and PPO, the model becomes an active "agent" playing a strategic game. Just like an AI learning to play Mario, it executes a "policy" to choose the best actions (words) to maximize its final reward.

RLHF: PPO -> ChatGPT



The three-step Reinforcement Learning from Human Feedback (RLHF) pipeline introduced by Ouyang et al. (2022) in the InstructGPT paper. This methodology is the foundational process that transformed raw text-prediction models like GPT-3 into highly aligned conversational agents like ChatGPT.

Step 1: Supervised Fine-Tuning (SFT)

- The Goal: Teach the base model the fundamental format of how to answer questions and follow instructions.
- The Process: A prompt is sampled from a dataset (e.g., "Explain the moon landing to a 6 year old"). A human labeler writes out the ideal, desired response. The base language model is then fine-tuned on this high-quality demonstration data using standard supervised learning.

Step 2: Training a Reward Model (RM)

- The Goal: Build a secondary AI system that can autonomously judge responses based on human preferences.

- The Process: A prompt is sampled, and the SFT model generates several different possible outputs (Outputs A, B, C, and D). A human labeler reads these outputs and ranks them from best to worst. This ranking data is used to train a completely separate "Reward Model" to predict which types of responses humans will prefer.

Step 3: Proximal Policy Optimization (PPO)

- The Goal: Automate the improvement of the main model using the Reward Model as a continuous, scalable judge.
- The Process: A new prompt is sampled, and the main model (now acting as a "policy") generates an output. The Reward Model evaluates this output and calculates a mathematical reward score. This reward is fed back into the main model using the PPO reinforcement learning algorithm, which updates the model's internal parameters to generate higher-reward answers in the future.

By Step 3, the AI is optimizing its behavior against a mathematical representation of human values, which allows it to improve vastly without requiring humans to manually write out millions of perfect training examples.

Part 3 - C

50

RLHF: DPO

- Idea: maximize probability of **preferred output**, minimize **the other**

$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

Reinforcement Learning from Human Feedback (RLHF)
x: "write me a poem about the history of jazz"
y_w > y_l
preference data → maximum likelihood → reward model → label rewards → LM policy → sample completions → reinforcement learning

Direct Preference Optimization (DPO)
x: "write me a poem about the history of jazz"
y_w > y_l
preference data → maximum likelihood → final LM

DPO
[Rafailov+ 2023]

- This is ~equivalent (same global minima) to RLHF/PPO
- Much simpler than PPO and performs as well => standard (in open source community)

Stanford 

DPO (Direct Preference Optimization), a 2023 breakthrough from Stanford researchers (Rafailov et al.) that made fine-tuning models on human preferences vastly simpler, faster, and cheaper.

1. The Core Idea: "Cut Out the Middleman"

In the previous slide on traditional RLHF with PPO, the process was complicated:

1. Collect human preference data.
2. Train a separate Reward Model (the judge).
3. Run an unstable Reinforcement Learning (PPO) loop where the main model generates text and the judge grades it.

Running an RL loop like PPO is notoriously difficult: it is unstable, consumes massive amounts of GPU memory (because multiple models must stay loaded simultaneously), and frequently crashes.

DPO asks a simple question:

Why bother training a separate Reward Model at all if our ultimate goal is just to make the language model produce good answers and avoid bad answers?

Instead of an RL loop, DPO directly increases the probability of the preferred output (y_w) and decreases the probability of the rejected output (y_l) in a single training step.

2. Looking at the Two Boxes

The diagram in the middle shows the architectural comparison:

- Left (Traditional RLHF):

Preference Data $\rightarrow$ Reward Model $\longleftrightarrow$ LM Policy via PPO Loop

Complex, slow, and requires juggling multiple neural networks at once.

- Right (DPO):

Preference Data $\rightarrow$ Direct Maximum Likelihood $\rightarrow$ Final LM

Clean, straightforward, and looks almost like standard supervised learning.

3. Reading the Math Without Fear

The equation at the top defines the DPO loss:

$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

Breaking down the variables:

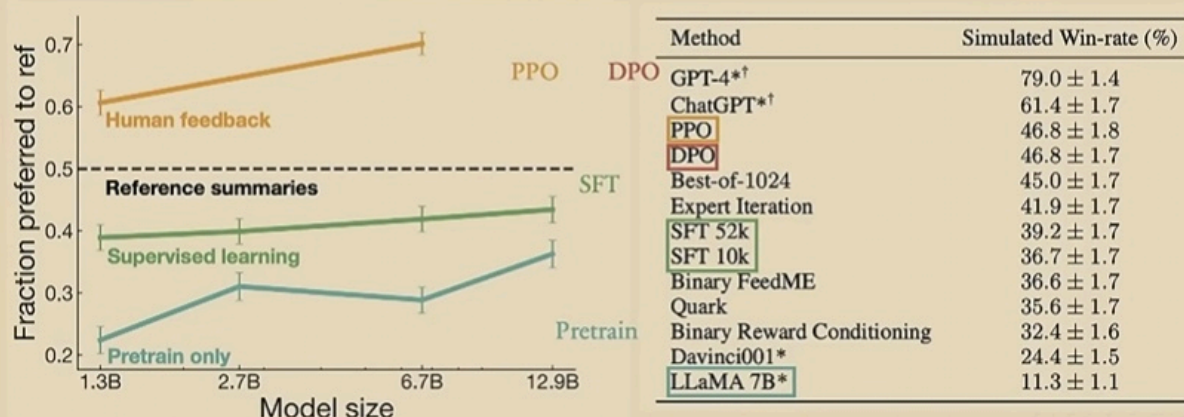
- x : The input prompt.
- y_w (Winner): The preferred answer (highlighted in red box).
- y_l (Loser): The rejected answer (highlighted in green box).
- π_{θ} : The model being trained.
- π_{ref} : The original reference model (acting as an anchor so the AI doesn't forget its core language skills or degenerate into gibberish).
- β (Beta): A knob controlling how tightly the model stays anchored to the reference model.

The formula simply calculates the gap between the score of the winning answer and the score of the losing answer, adjusting the weights so that the good answer becomes much more likely than the bad one.

4. Why DPO Became the Open-Source Standard

- Mathematically Equivalent: The paper proved that DPO reaches roughly the exact same optimal solution (the same global minimum) as the complex PPO process.
- No Reinforcement Learning Headaches: DPO converts a messy reinforcement learning problem into a stable, straightforward classification problem.
- Massive Resource Savings: Because you do not need to host a separate reward model or generate samples on the fly, developers and researchers with limited GPU resources can easily align models at a fraction of the hardware cost.

RLHF: gains



Learn to summarize
[Stiennon+ 2020]

Stanford
AlpacaFarm
[Dubois+ 2023]



The massive performance gains achieved by Reinforcement Learning from Human Feedback (RLHF), specifically comparing PPO and DPO against traditional supervised learning (SFT).

1. The Left Graph: The Power of Human Feedback

- What it shows: This graph, based on a 2020 paper by Stiennon et al., compares three different training methods on a summarization task. The Y-axis measures how often a human preferred the model's summary over the original human reference summary.
- Pretrain only (Bottom Line): A raw, pre-trained model performs poorly, hovering between a 20% and 36% win rate.
- Supervised learning / SFT (Middle Line): Fine-tuning the model using standard supervised learning improves performance, but it plateaus around 40% and never beats the human reference (the dotted 0.5 line).
- Human feedback / PPO (Top Line): Training the model using RLHF causes a massive jump in quality. It easily crosses the 0.5 threshold, meaning the AI's summaries are actually preferred more often than the original human-written summaries.
- The Key Takeaway: The graph proves that a tiny 1.3 Billion parameter model trained with human feedback is significantly better than a massive 12.9 Billion parameter model trained only with supervised learning. RLHF allows developers to extract vastly superior performance out of much smaller models.

2. The Right Table: DPO vs. PPO

- What it shows: This table, sourced from the 2023 AlpacaFarm simulation framework, ranks different training methods based on their simulated win-rate against a baseline.
- The Baseline: The raw pre-trained LLaMA 7B model starts at the very bottom with an 11.3% win rate.
- SFT Gains: Standard Supervised Fine-Tuning (SFT 52k) brings the win rate up to 39.2%.
- The RLHF Leap: Both PPO and DPO boost the model's win rate to an identical 46.8%.
- The Key Takeaway: As discussed in the previous slide, DPO was designed to be much simpler to implement than the complex PPO algorithm. This table provides the empirical proof that DPO's simplified math does not sacrifice any performance; it matches PPO's 46.8% win rate exactly while being much easier for researchers to run.

RLHF: human data

• Data: human crowdsourcing

In this task, you will be provided with a Prompt from a user (e.g., a question, instruction, statement) to an AI chatbot along with two potential machine-generated Responses to the Prompt. Your job is to assess which of the two Responses is better for the Prompt, considering the following for each Response:

Helpfulness: To what extent does the Response provide useful information or satisfying answers for the Prompt? Responses should: - Address the intent of the user's Prompt such that a user would not feel the Prompt was ignored or misinterpreted by the Response. - Provide specific, comprehensive, and up-to-date information for the user needs expressed in the Prompt. - Be verifiable and unbiased. The response should not contain any sensational information or contradict itself across sentences (e.g., refer to two different people with the same name as if they are the same person). - Adhere to any requirements indicated in the Prompt such as an explicitly specified word length, tone, format, or information that the Response should include. - Not contain inaccurate, deceptive, or misleading information based on your current knowledge or quick web search - you do not need to perfect a rigorous fact check. - Not contain harmful, offensive, or overly sexual content. A Response may occasionally intentionally avoid or decline to address the question/topic of the Prompt and may provide a reason for why it is unable to respond. For example, "Sorry, there may not be a helpful answer to this question." These responses can be considered helpful in cases where an appropriate helpful response to the Prompt does not seem possible. Presentations: To what extent is the content of the Response conveyed well? Responses should: - Be organized in a structure that is easy to consume and understand, flowing in a logical order and makes good use of formatting such as paragraphs, lists, or tables. - Be clearly written in a polite casual tone that is engaging, direct, and inclusive. The tone should not be overly formal, overly academic, sales, or judgmental in a way that most users would consider to be off putting or annoying. - Have consistent style with natural phrasing and transitions as if composed by a single natural person. - Not be rambling, repetitive, or contain clearly off-topic information. (On-topic information should not be repeated multiple times. It is helpful for users to consume the helpful information in a response if there is repetitive or less helpful information mixed into the response.) - Not include notable language issues or grammatical errors.	Rating scale: Not at All Helpful: Response is irrelevant/unrelated, contains even a single piece of nonsensical/hallucinated/misleading information, and/or contains harmful/offensive/offensive sexual content. Slightly Helpful: Response is somewhat related to the Prompt, does not address important aspects of the Prompt, and/or contains outdated information. Barely Helpful: Response partially addresses the intent of the Prompt (most users would avoid more information), contains some outdated information, and/or is lacking helpful details/facts. Very Helpful: Response addresses the intent of the Prompt with a satisfying response. Some users might want a more comprehensive response with additional details or context. It is comparable to a response an average human with basic subject-matter knowledge might provide. Extremely Helpful: Response noticeably addresses the intent of the Prompt and provides helpful details/context. It is comparable to a response a knowledgeable informed human with subject-matter expertise might provide.
Rating scale: Poor: Response is poorly written or has notable structural, formatting, language, or grammar issues. Or Response has an awkward or inappropriate tone. Or the Response repeats entire information. Or any a small portion of the Response contains helpful information. Adaptable: Response could have been rephrased/generates better or may have more language/grammar issues. A minimal amount of less helpful information may be present. Users would still find the content of the Response was easy to consume. Excellent: Response is very well written and organized. Sentences flow in a logical order with smooth transitions and consistent style. The content of the Response is conveyed in a way that is comparable to a response a skilled human might provide.	guidelines

Overall, you should consider both factors in your SFT rating of which response is better. A more concise response presenting the most helpful information directly and clearly is usually better than a longer response that may be better to consume and/or contains clearly off-topic information. Responses with Poor Presentation (e.g., rambling, inappropriate tone) should play a significant role in your assessment of which side is better. It may help to imagine the user chatting with a real person and consider which Response most users would prefer to reading from a real person.

► Annotation Guidelines

Instruction

Tell me about self driving cars

Input

example

AI Response 1

Self driving cars are vehicles that are capable of detecting their surroundings and navigating without any input from a human driver. They use a variety of sensors, such as cameras, radar, and lidar, to detect objects in their environment and use advanced algorithms to make decisions about how to maneuver around them. Self driving cars are becoming increasingly popular and are being used in a variety of applications, such as ride-hailing services and autonomous delivery services.

AI Response 2

Self driving cars are cars that are equipped with sensors, cameras, and software, which allows them to navigate without the need for a driver. Self driving cars are able to monitor their environment and navigate without the assistance of a driver. Self driving cars use a combination of technologies such as GPS, radar, lidar, and computer vision to detect their environment and avoid obstacles. They are also able to update their maps and adjust their routes to account for variables such as traffic and construction.

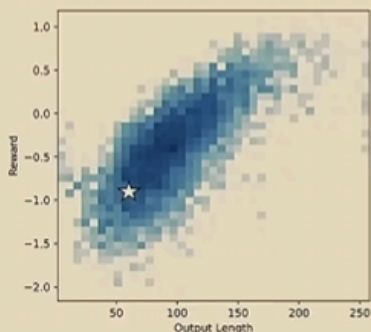
The main benefit of self driving cars is their ability to reduce crashes and save lives. Autonomous vehicles have the potential to save up to 10,000 lives annually. They are also more energy efficient than traditional vehicles, as they do not require a driver who has to maintain a minimum speed. Autonomous vehicles also create new opportunities for increased mobility, allowing those who are unable to drive to get around.

Rating

- ☐ Response 1 is better.
- ☐ Response 1 is only slightly better. (only pick this if it's truly close)
- ☐ Response 2 is only slightly better. (only pick this if it's truly close)
- ☐ Response 2 is better.

RLHF: challenges of human data

- Slow & expensive
- Hard to focus on correctness rather than form (eg length)



Question: Why don't adults roll off the bed?

☆ SFT (Before); 59 tokens

Adults typically do not roll off of the bed because they have developed the muscle memory to keep their bodies from involuntarily moving during sleep and maintaining proper posture.

RLHF (After); 243 tokens: Similar output, but much longer / more details

Adults generally do not roll off of the bed because they have developed muscle control and awareness which helps them to remain upright while sleeping. Additionally, most adults find it uncomfortable or even painful to move around in their sleep, so rolling off the bed would be difficult without waking up first. There may also be safety concerns such as falling out of bed and potentially hurting oneself if one were to lose balance or fall from the ...

Long way to go
[Singhal+ 2024]

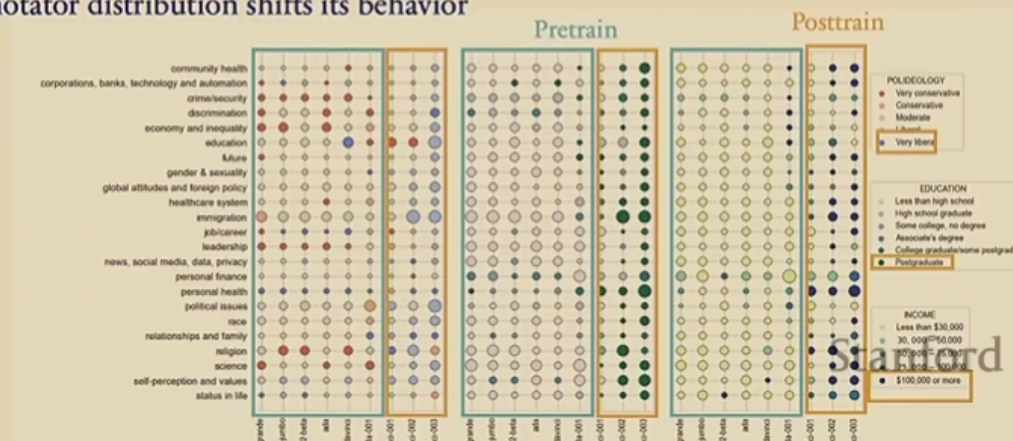
Stanford



RLHF: challenges of human data

- Slow & expensive
- Hard to focus on correctness rather than form (eg length)
- Annotator distribution shifts its behavior

LLM Opinion
[Santurkar+ 2022]



RLHF: challenges of human data

- Slow & expensive
- Hard to focus on correctness rather than form (eg length)
- Annotator distribution shifts its behavior
- Crowdsourcing ethics

TECHNOLOGY

AMERICA ALREADY HAS AN AI UNDERCLASS

Search engines, ChatGPT, and other AI tools wouldn't function without an army of contractors. Now those workers say they're underpaid and mistreated.

By Matteo Wong

Stanford



How human data is collected for RLHF and the four major flaws and challenges of relying on human feedback.

How Human Data is Collected (Crowdsourcing)

To train the Reward Model, companies pay human crowdworkers (contractors) through platforms like Scale AI or Surge AI.

- Detailed Guidelines: Humans receive multi-page rulebooks defining what makes a response helpful, polite, and safe.
- Side-by-Side Grading: Workers are given a prompt (e.g., *"Tell me about self driving cars"*) alongside two different AI responses (Response 1 vs. Response 2).
- Binary/Likert Rating: The worker selects which response is better (e.g., *"Response 2 is better"*). This preference data is then fed directly into the Reward Model or DPO loss.

The 4 Major Challenges of Human Data

1. Form Over Correctness (Length Bias)

- The Problem: Humans are naturally biased toward believing longer, well-formatted answers are smarter—even when they are wordy or slightly wrong.
- The Graph (Singhal et al., 2024): The scatter plot shows that as output token length increases along the horizontal axis, the reward score on the vertical axis shoots straight up.
- Resulting Behavior: The model learns to "game" the system. In the example shown, the SFT model answered the question *"Why don't adults roll off the bed?"* in a concise 59 tokens. After RLHF, the model learned that longer answers get higher ratings, inflating the response to 243 tokens with unnecessary filler.

2. Annotator Demographics Shift Model Behavior

- The Problem: AI models mirror the values, opinions, and worldviews of the specific people hired to rate them.
- The Study (Santurkar et al., 2023): When researchers analyzed model opinions across controversial topics, they found that post-training systematically shifted models toward the specific demographic profile of the annotators—typically skewing more liberal, highly educated (postgraduate), and higher-income than the general global population.

3. Slow & Expensive

- Labeling millions of complex pairwise comparisons (especially for specialized domains like software engineering, law, and medicine) requires enormous capital and weeks of manual turnaround time.

4. Crowdsourcing Ethics

- As shown in *The Atlantic* headline ("*America Already Has an AI Underclass*" by Matteo Wong), much of the raw labeling and safety moderation work relies on low-wage contractors worldwide who are tasked with reviewing toxic, violent, or distressing prompts with minimal support.

Part 4

56

Evaluation: aligned LLM

- How do we evaluate something like ChatGPT?
- Challenges:
 - Can't use validation loss to compare different methods
 - Can't use perplexity: not calibrated Some aligned LLMs are policies!
 - Large diversity
 - Open-ended tasks => hard to automate
- Idea: ask for annotator preference between answers

Table 1: Distribution of use case categories from our API prompt dataset.

Use-case	(%)
Generation	45.6%
Open QA	12.4%
Brainstorming	11.2%
Chat	8.4%
Rewrite	6.6%
Summarization	4.2%
Classification	3.5%
Other	3.5%
Closed QA	2.6%
Extract	1.9%

InstructGPT[®]
[Ouyang+ 2022]



Evaluating an aligned chatbot like ChatGPT is fundamentally harder than evaluating a pre-trained model because open-ended conversational tasks rarely have a single "correct" answer key.

1. The Core Problem

During pre-training, evaluation is straightforward: you measure how well the model predicts the next word (perplexity) or test it on multiple-choice exams with fixed answer keys (like MMLU). But once a model is post-trained into an assistant, users ask open-ended questions where automated grading scripts fail.

2. The 4 Key Challenges

- Cannot use validation loss: Different alignment techniques (SFT, PPO, DPO) optimize completely different loss functions. Because their mathematical training objectives differ, comparing their raw validation loss numbers is like comparing apples to oranges.
- Cannot use perplexity (Not calibrated): Perplexity measures how closely a model mimics standard web text distributions. Aligned models trained with reinforcement learning act as goal-directed policies—they are trained to produce helpful, polite actions, not to replicate average human writing distributions.

- Massive diversity of real-world use: The table from the InstructGPT paper (Ouyang et al., 2022) shows what real users actually submit to APIs:
 - Generation (45.6%): Story writing, drafting emails, code generation.
 - Open QA (12.4%) & Brainstorming (11.2%): Exploratory questions with countless valid answers.
 - Chat (8.4%), Rewriting (6.6%), Summarization (4.2%): Highly subjective tasks with varying user standards.
- Open-ended tasks resist automation: If a user asks, *"Brainstorm three creative startup ideas for eco-friendly packaging,"* there is no objective answer key. A script cannot verify whether an idea is "creative" or "practical."

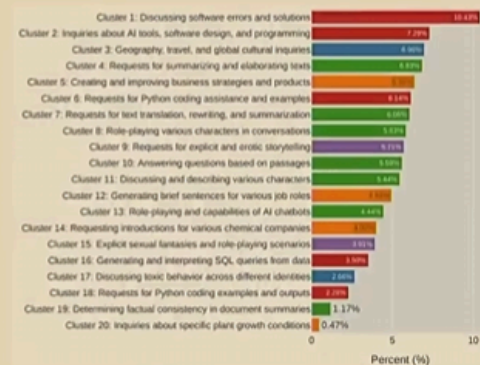
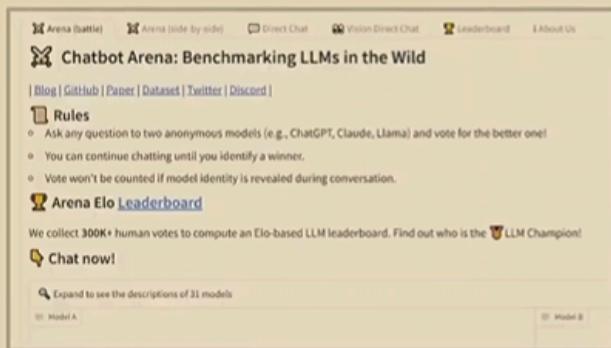
3. The Solution: Pairwise Annotator Preference

Because automated metrics fail on open-ended text, the industry standard is pairwise human evaluation:

- Present two different models with the exact same user prompt.
- Have human judges blind-review both outputs side by side.
- Tally the votes to determine a head-to-head win rate (the foundational concept behind platforms like LMSYS Chatbot Arena).

Human evaluation: eg ChatBot Arena

- Idea: have users interact (blinded) with two chatbots, rate which is better.



- Problem: cost & speed!

Chatbot Arena, which is the gold standard for testing how good AI models are when real people use them in the real world.

Instead of grading models on standard multiple-choice tests, Chatbot Arena treats evaluation like a blind taste test (or a competitive video game ladder).

1. How It Works (The Blind Battle)

- Anonymous Matchup: You visit the website and type in any prompt or question you want.
- Side-by-Side Generation: Two different anonymous models (for example, GPT-4 and Claude 3.5 Sonnet) generate responses side by side as "Model A" and "Model B."
- Blind Voting: You read both responses and vote on which one was better, more helpful, or more accurate.
- The Reveal & Elo Score: Only *after* you cast your vote are the model identities revealed. Every vote updates the models' Elo rating—the exact same mathematical ranking system used in chess and competitive gaming.

2. Why It Matters (The Bar Chart on the Right)

Traditional benchmarks test whether an AI can pass a college biology or history test. But that's not how people actually use AI.

The chart on the right analyzes what people in the "wild" are actually asking:

- The top categories are practical developer tasks: debugging software errors, system design, and writing Python scripts.
- Other heavy categories include summarizing long text, drafting business strategies, and creative role-playing.

Because real-world prompts are open-ended, messy, and creative, you cannot write an automated grading script with an answer key. Chatbot Arena works because it captures how well models handle the actual messy tasks humans do every day.

3. The Catch: Cost & Speed

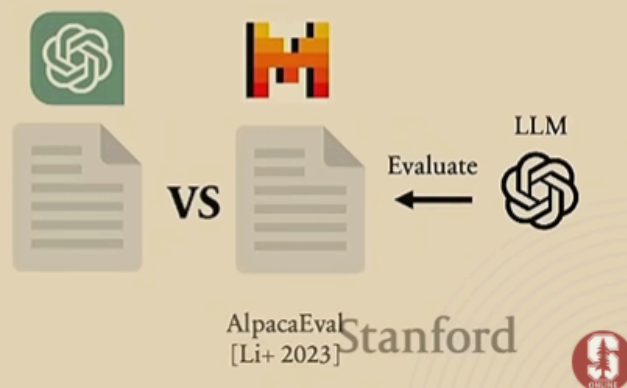
While human evaluation is the most trustworthy metric we have, it has two major drawbacks:

- It is slow: Collecting hundreds of thousands of human votes takes weeks or months.
- It is expensive: You cannot use real human battles in the middle of a training run to quickly check if an engineering tweak improved your model by 2%.

This limitation is why researchers eventually developed "LLM-as-a-Judge" (using a massive model like GPT-4 to simulate human voters), which balances the nuance of open-ended evaluation with the speed of automated testing.

LLM evaluation: eg AlpacaEval

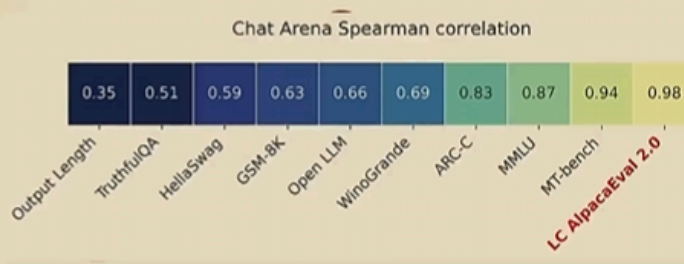
- Idea: use LLM instead of human
- Steps:
 - For each instruction: generate output by baseline and model to eval
 - Ask GPT-4 which output is better
 - Average win-probability $\Rightarrow$ win rate



LLM evaluation: eg AlpacaEval

- Idea: use LLM instead of human
- Steps:
 - For each instruction: generate output by baseline and model to eval
 - Ask GPT-4 which output is better
 - Average win-probability $\Rightarrow$ win rate

- Benefits:
 - 98% correlation with ChatBot Arena
 - < 3 min and < \$10
- Challenge: **spurious correlation**



AlpacaEval [Li+ 2023] Stanford



AlpacaEval and the concept of "LLM-as-a-Judge"—the solution researchers created to get the nuanced grading of human evaluation without the high financial cost or slow turnaround times.

1. The Core Concept: LLM-as-a-Judge

Instead of paying crowdsourced human annotators to read and rank thousands of chatbot answers, researchers use a frontier model (like GPT-4) to act as the automated referee.

The Pipeline:

1. Side-by-Side Generation: For a given test prompt, generate two answers: one from a fixed baseline model (such as GPT-3.5 or an earlier checkpoint) and one from the model being evaluated.
2. GPT-4 Evaluates: Feed the prompt along with both outputs into GPT-4 and instruct it to pick the superior response.
3. Calculate Win Rate: Average the win probability across all test prompts to determine the model's overall win rate percentage.

2. Why It Was a Breakthrough

- High Human Agreement: AlpacaEval achieved a 0.98 (98%) Spearman correlation with the human votes on LMSYS Chatbot Arena. This proved that a strong model's judgment almost perfectly mirrors human preferences.
- Speed & Cost: Running a full human evaluation on thousands of prompts can cost thousands of dollars and take weeks. AlpacaEval can evaluate a model in under 3 minutes for less than \$10 in API tokens.

3. The Flaw: Spurious Correlation & Verbosity Bias

Just like human evaluators, LLM judges have cognitive blind spots. The most severe issue is length bias (verbosity bias): the model judge frequently mistakes sheer word count for quality.

- Gaming the System: If a model pads its responses with repetitive bullet points and verbose explanations, the judge model gives it a higher rating—even if the shorter answer was crisper and more accurate.
- The Raw Numbers: In the left table (*AlpacaEval*), gpt-4-1106-preview scores only 22.9% when generating concise answers, but jumps to 64.3% simply by adopting a verbose tone.

4. The Fix: Length-Controlled AlpacaEval (LC AlpacaEval)

To prevent developers from artificially inflating their leaderboard rank by just making their models talk more, researchers introduced Length-Controlled AlpacaEval (Dubois et al., 2023):

- Uses regression and causal inference to mathematically isolate and neutralize the impact of response length.
- Scores responses based on content quality regardless of length. As shown in the right table (*Length-controlled AlpacaEval*), the scores between concise (41.9%) and verbose (51.6%) responses normalize dramatically.

LLM evaluation: spurious correlation

- e.g. LLM prefers longer outputs

	AlpacaEval		
	concise	standard	verbose
gpt4_1106_preview	22.9	50.0	64.3
Mixtral-8x7B-Instruct-v0.1	13.7	18.3	24.6
gpt4_0613	9.4	15.8	23.2
claude-2.1	9.2	15.7	24.4
gpt-3.5-turbo-1106	7.4	9.2	12.8
alpaca-7b	2.0	2.6	2.9

AlpacaEval LC
[Dubois+ 2023]



LLM evaluation: spurious correlation

- e.g. LLM prefers longer outputs
- Possible solution: regression analysis / causal inference to “control” length

	AlpacaEval			Length-controlled AlpacaEval		
	concise	standard	verbose	concise	standard	verbose
gpt4_1106_preview	22.9	50.0	64.3	41.9	50.0	51.6
Mixtral-8x7B-Instruct-v0.1	13.7	18.3	24.6	23.0	23.7	23.2
gpt4_0613	9.4	15.8	23.2	21.6	30.2	33.8
claude-2.1	9.2	15.7	24.4	18.2	25.3	30.3
gpt-3.5-turbo-1106	7.4	9.2	12.8	15.8	19.3	22.0
alpaca-7b	2.0	2.6	2.9	4.5	5.9	6.8

AlpacaEval LC
[Dubois+ 2023]

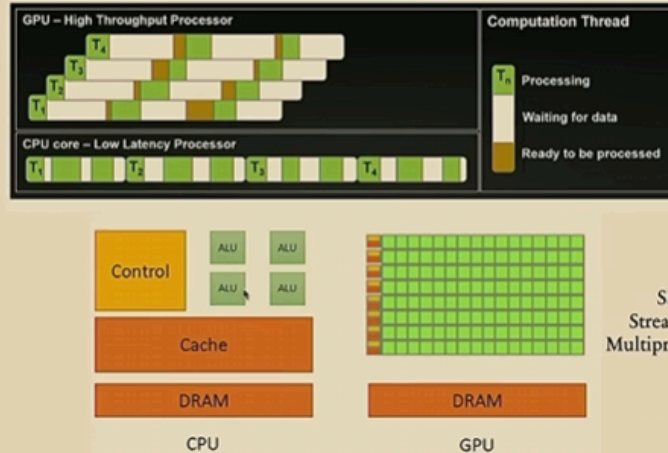


Part 5

62

Systems 101: GPUs

- Massively parallel: same instruction applied on all thread but different inputs.
=> Optimized for throughput!



SM
Streaming
Multiprocessors

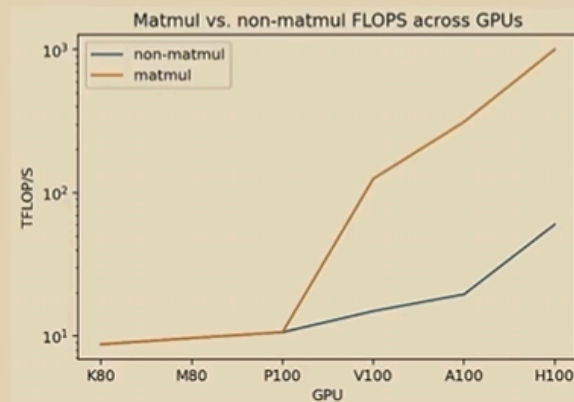
Stanford



63

Systems 101: GPUs

- Massively parallel
- Fast matrix multiplication: special cores >10x faster than other fp ops

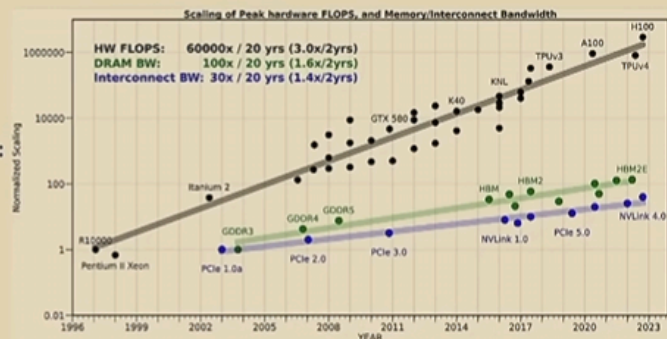


Stanford



Systems 101: GPUs

- Massively parallel
- Fast matrix multiplication
- Compute > memory & communication:
 - Hard to keep processors fed with data



BERT transformer

Table 1. Proportions for operator classes in PyTorch.

	Operator class	% flop	% Runtime
Matmul	△ Tensor contraction	99.80	61.0
	□ Stat. normalization	0.17	25.5
Activation	○ Element-wise	0.03	13.5

DataMovement
[Ivanov+ 2020]

Stanford



Systems: low precision

- Fewer bits => faster communication & lower memory consumption
- For deep learning: decimal precision ~doesn't matter except exp & updates
 - Matrix multiplications can use bf16 instead of fp32
- For training: **Automatic Mixed Precision (AMP)**
 - Weights stored in fp32, but before computation convert to bf16
 - Activation in bf16 => main memory gains
 - (Only) matrix multiplication in bf16 => speed gains
 - Gradients in bf16 => memory gains
 - Master weights updated fp32 => full precision

Stanford



Systems: operator fusion

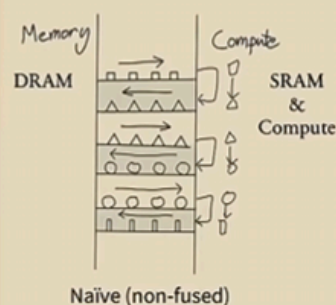
- Problem:

- communication is slow

```
x1 = x.cos() # Read from x in global memory, write to x1
x2 = x1.cos() # Read from x1 in global memory, write to x2
```

- every new PyTorch line moves variables to global memory

- Idea: communicate once



Stanford



Outlook

Haven't touched upon:

- | | |
|---------------------------|-------------------------------|
| • Architecture: MoE & SSM | • Misuse |
| • Decoding & inference | • Context size |
| • UI & tools: ChatGPT | • Data wall |
| • Multimodality | • Legality of data collection |

Going further:

- CS224N: more of the background and historical context. Some adjacent material.
- CS324: more in-depth reading and lectures.
- CS336: you actually build your LLM. Heavy workload!

Stanford



Primary Video Reference

- **Stanford Online. (2024, August 27).** *Stanford CS229 | Machine Learning | Building Large Language Models (LLMs)*. YouTube. <https://www.youtube.com/watch?v=9vM4p9NN0Ts>

Supervised Fine-Tuning (SFT)

- **Köpf et al. (2023):** *OpenAssistant*. Cited as the framework for collecting high-quality human conversational data for initial model tuning.
- **Taori et al. (2023):** *Alpaca*. Cited for demonstrating how to scale instruction-following data cheaply by using a larger LLM to generate seed tasks.
- **Zhou et al. (2023):** *LIMA (Less Is More for Alignment)*. Cited to prove that SFT requires very little data (a few thousand examples) because it only teaches the model formatting, not new knowledge.

Reinforcement Learning from Human Feedback (RLHF) & Alignment

- **Bradley & Terry (1952):** *Rank analysis of incomplete block designs*. The foundational 1952 mathematical formula used to classify human preferences into continuous scores for the Reward Model.
- **Stiennon et al. (2020):** *Learning to summarize from human feedback*. Cited to show the empirical performance gains where small models trained with human feedback outperform massive models trained only with supervised learning.
- **Ouyang et al. (2022):** *InstructGPT*. The foundational paper outlining the complete three-step RLHF pipeline (SFT $\rightarrow$ Reward Model $\rightarrow$ PPO) that led to ChatGPT.
- **Rafailov et al. (2023):** *Direct Preference Optimization (DPO)*. Cited for introducing a mathematically equivalent but vastly simpler alternative to PPO that eliminates the need for a separate Reward Model.
- **Dubois et al. (2023):** *AlpacaFarm*. Cited for simulating win rates that prove DPO achieves the exact same performance gains (46.8%) as PPO.

Human Data Challenges & System Evaluation

- **Santurkar et al. (2023):** *Whose Opinions Do Language Models Reflect?* Cited to demonstrate how an LLM's opinions and behaviors shift to match the specific demographics of the human annotators.
- **Wong, M. (The Atlantic):** *"America Already Has an AI Underclass."* Cited to highlight the ethical and labor challenges of relying on crowdsourced contractors.

- **Singhal et al. (2024):** *A Long Way to Go*. Cited for proving the "verbosity bias" where RLHF accidentally teaches models that longer answers are better, even when unnecessary.
- **Chiang et al. (2023):** *Chatbot Arena*. Cited as the gold standard for blind, pairwise human evaluation and Elo ranking.
- **Li et al. (2023):** *AlpacaEval*. Cited for introducing the "LLM-as-a-judge" methodology to automate evaluation cheaply.
- **Dubois et al. (2023):** *Length-Controlled (LC) AlpacaEval*. Cited for using regression analysis to neutralize the spurious length correlation in automated LLM judges.